DMS Connectivity Pre-Flight Checklist

Topology: on-prem PostgreSQL → Cloud WAN → isolated VPC (no IGW/NAT) → DMS replication instance in database subnets → private Aurora PostgreSQL.

Work top to bottom — ordered by what breaks first.

---

0. Account prerequisites

- ☐ **`dms-vpc-role` exists** — `aws iam get-role --role-name dms-vpc-role` *Missing: the replication subnet group fails to create, with an error naming neither the role nor the fix.*
- ☐ **`dms-cloudwatch-logs-role` exists** — `aws iam get-role --role-name dms-cloudwatch-logs-role` *Missing: tasks run, logging silently does nothing, and a failure has no detail behind it.*

Fixed names, one set per account. Not created by the module by default.

---

1. Subnets and network interfaces

- ☐ Replication subnet group spans **at least 2 subnets in different AZs** (DMS requires this even for single-AZ)
- ☐ **All** replication instance private IPs captured — `terraform output -json replication_instance_private_ips`
- ☐ Sufficient free IPs in each subnet for the ENIs

> ⚠️ **Multi-AZ places one ENI per subnet.** The on-prem firewall must allow **every** address, not just the first. Opening only one works until the first AZ failover, then looks like a random network fault.

---

2. Cloud WAN routing

- ☐ VPC attachment is in the **correct segment**
- ☐ On-prem prefix **propagates** into the database subnets' route table
- ☐ **Return path**: on-prem routes the VPC CIDR back over the **same** attachment
- ☐ Core network **segment policy** permits the traffic (a route is not a policy)
- ☐ MTU / fragmentation verified end to end

> ⚠️ **Asymmetric routing is the worst failure here.** `test-connection` uses one small exchange and **passes**. The full load then stalls part-way. If you see "connection test fine, load stuck at 30%" — look here first.

---

3. DNS resolution

- ☐ On-prem hostname **resolves from inside the VPC** — from an EC2 instance in the same subnet: `dig +short <onprem-host>`

- ☐ Route 53 Resolver **outbound endpoint** exists — `aws route53resolver list-resolver-endpoints`
- ☐ **Forwarding rule** to on-prem DNS exists **and is associated with this VPC** — `aws route53resolver list-resolver-rule-associations`
- ☐ Resolver endpoint security group allows **53 TCP and UDP** to the on-prem resolvers

> ⚠️ A name that resolves only on-premises produces a connection timeout that **looks exactly like a firewall problem**. **Quick triage:** temporarily put the IP instead of the hostname in the secret. If it connects, the fault is DNS, not the network.

---

4. VPC endpoints (mandatory — there is no internet path)

Endpoint	Type	Required when
`secretsmanager`	Interface	Always (credentials via `secret_arn`)
`kms`	Interface	CMK-encrypted secret, or SSE-KMS S3 target
`logs`	Interface	Always (task logging)
`monitoring`	Interface	**Verify — see below**
`s3`	**Gateway**	Free. Add it regardless

For **every** interface endpoint:

- ☐ **Private DNS enabled** *Without it DMS resolves the public name and the task **hangs** rather than erroring.*
- ☐ Endpoint's **own security group allows 443 inbound from the replication instance's SG**
- ☐ Endpoint present in **both** AZs

> ⚠️ "No ingress required" in the network contract refers to the **replication instance**. The **endpoints** absolutely need inbound 443. This is misread constantly.

`monitoring` is unresolved — run this once the first task is running:

```
aws cloudwatch list-metrics --namespace AWS/DMS \
  --query 'Metrics[?MetricName==`CDCLatencyTarget`].Dimensions' --output
json
```

Empty while a task is live ⇒ the endpoint **is** needed, **and** your CDC latency alarms are sitting in ALARM permanently (they treat missing data as breaching).

---

5. Security groups — three distinct paths

```
[DMS SG] --egress 5432--> [Aurora SG]         needs ingress 5432 from DMS
SG
[DMS SG] --egress 5432--> on-prem via Cloud WAN   on-prem firewall allows
```

```
ALL ENI IPs
[DMS SG] --egress 443---> [Endpoints SG]      needs ingress 443 from DMS SG
```

- [ ] DMS SG: egress 5432 to on-prem CIDR
- [ ] DMS SG: egress 5432 to target
- [ ] DMS SG: egress 443 to VPC endpoints
- [ ] **Aurora SG: ingress 5432 from the DMS SG** ← frequently missed because DMS sits in the same subnets
- [ ] Endpoints SG: ingress 443 from the DMS SG
- [ ] DMS SG: **no ingress rules needed at all**

---

6. PostgreSQL source (on-prem)

- [ ] `wal_level = logical` — `SHOW wal_level;`
- [ ] `max_replication_slots` > number of concurrent tasks
- [ ] `max_wal_senders` sufficient
- [ ] DMS user has the `REPLICATION` attribute
- [ ] `pg_hba.conf` allows **all** replication instance ENI IPs, for both normal and `replication` connections
- [ ] **A plan exists to drop the replication slot after cutover** — `SELECT * FROM pg_replication_slots;`

> ⚠️ An inactive replication slot pins WAL **forever**. The source's disk fills — an outage on the database you were migrating *away from*. The module enables a heartbeat to keep the slot advancing; dropping it afterwards is yours.

---

7. PostgreSQL target (Aurora)

- [ ] Application schema already created — this is **pre-data**; DMS creates nothing under `DO_NOTHING`
- [ ] `dms_control` schema exists, or the DMS user can create it
- [ ] DMS user has CREATE rights on the target schemas
- [ ] Primary-key collation checked (non-`C` collation can cause false validation mismatches; the module sets `HandleCollationDiff = true`)
- [ ] Secret JSON has the shape DMS expects: `username`, `password`, `host`, `port`, `dbname`

---

8. Final verification

```
aws dms test-connection \
  --replication-instance-arn <instance-arn> --endpoint-arn <endpoint-arn>

aws dms describe-connections \
  --filters Name=replication-instance-arn,Values=<instance-arn> \
  --query 'Connections[].[EndpointIdentifier,Status,LastFailureMessage]' --
output table
```

Result	Where to look
Both `successful`	Proceed — but §2 return path is **not** proven by this test
Timeout on source	§2 routing, or §3 DNS
`could not translate host name`	§3 — DNS, definitively
Authentication failed	Secret JSON shape, or §6 `pg_hba.conf`
Timeout on target	§5 — Aurora SG ingress from the DMS SG
Secret / access denied error	§4 — `secretsmanager` endpoint, or `kms:Decrypt` on the secret's key

Top 3 to check before anything else

1. **On-prem return route to the VPC CIDR** (§2) — the only item that passes the connection test and still breaks the load.
2. **Private DNS on every interface endpoint** (§4) — produces a hang, not an error.
3. **All ENI IPs on the on-prem firewall** (§1) — works until the first AZ failover.

Machine-readable versions of most of this come straight from the module:

```
terraform output -json network_contract      # endpoints, DNS, return path,
notes
terraform output -json source_prerequisites  # the PostgreSQL source items
in §6
terraform output -json account_prerequisites # §0
```